Limitations of Array:-

1. Fixed in size
2. Homogeneous
3. No readymethods for utility operations

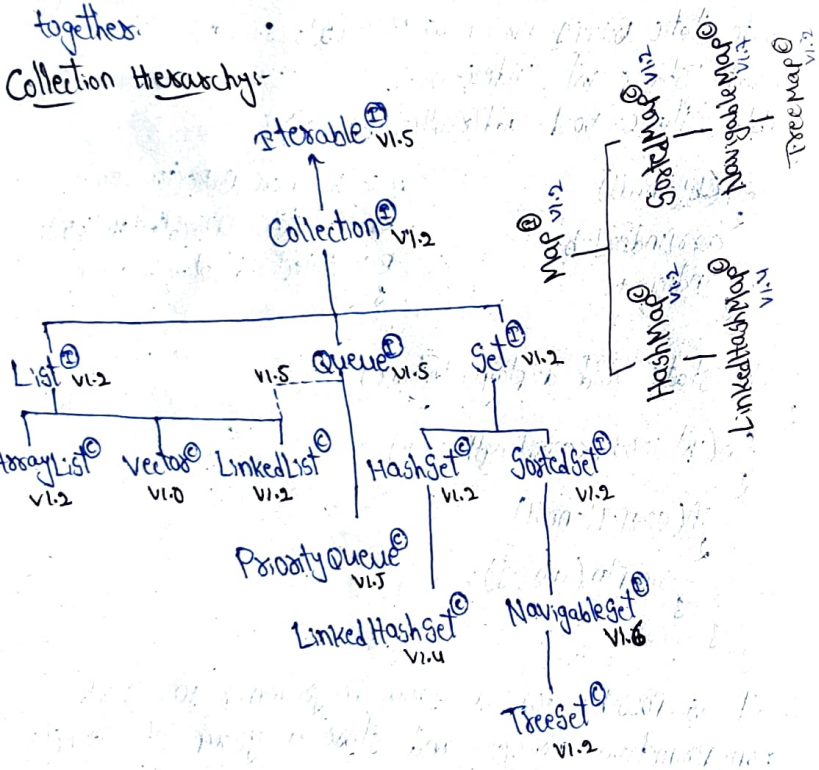
→ To overcome these limitations of array, the java library Provides Collection framework.

COLLECTION FRAMEWORK LIBRARY

→ The collection framework library is a group of Predefined Programs in the java library that is Present in java.util Package.

→ The collection framework library Provides several readymade containers with different properties to store Objects together.

Collection Hierarchy:-



Name — ArrayList

child of → List

Based on Datastructure → Resizable Array/ Growable Array

Best suited operation → Retrieval of elements.

Heterogeneous	Duplicates	Insertion order	Null Acceptance
Yes	Yes	Yes	Yes

Default Capacity — 10

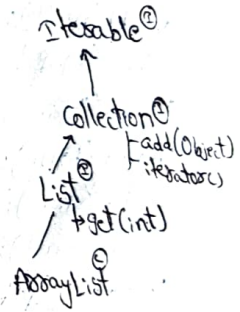
New capacity — (Old capacity * 3/2)+1

Marker Interface — Serializable, Cloneable, RandomAccess

Version — V1.2

Package — java.util

```
Public class MainClass
{
  psvm(){
  ArrayList al=new ArrayList();
   al.add("Java");
   al.add("JSPiders");
   al.add(23);
   al.add(new Person(20,"Renu"));
   al.add(null);
   al.add(new Employee(420,415));
   al.add("Java");
   al.add(null);
   al.add(new Bike(150,45.0));
   al.add(new Product(2235,50000));
   al.add("SQL");
  Iterator itr=al.iterator();
  while(itr.hasNext()){
  S.O.println(itr.next());
```



Name	Vector
Child of	List
Based on Datastructure	Resizable Array/Growable Array
Best Suited operation	Retrieval of elements
Heterogeneous yes	Null Acceptance yes
Duplicates yes	Default capacity 10
Insertion order yes	New capacity (old capacity * 2)
Marker Interface	Serializable, cloneable, RandomAccess
Version	V1.0
Package	java.util

```
Public class MainClass
{
  psvm(){
  Vector v = new Vector();
  v.add(new Mobile("IPhone",1.8));
   v.add(new Bike(300,15.0));
   v.add(null);
   v.add("JSpiders");
   v.add("Java Full stack");
   v.add(true);
   v.add(3.45);
   v.add(null);
   v.add("JSpiders");
   v.add(new Object()); // java.lang
   v.add(new Person(20,"Joy"));
   Iterator itr = v.iterator();
   while(itr.hasNext()){
      S.o.pln(itr.next());
   }
}
```

ArrayList	Vector
V1.2	V1.0
Non Legacy class	Legacy class
NC = (oc * 3/2) + 1	NC = (oc * 2)
Not Thread Safe	Thread Safe
Not Synchronized	Synchronized
Fast	Slow
Less Secure	More Secure

Name	LinkedList
Child of	List, Queue
Based on Data Structure	Doubly Linked List
Best Suited operation	Insertion and Deletion

Heterogeneous	Duplicates	Insertion order	Null Acceptance
Yes	Yes	Yes	Yes

Marker Interface - Serializable, Cloneable
Version - V1.2
Package - java.util

```
Public class MainClass
{
  Psvm(){
  LinkedList ll=new LinkedList();
   ll.add("java");
   ll.add("SQL");
   ll.add("j2ee");
   ll.add("j2ee");
   ll.add(null);
   ll.add(null);
   ll.add(new Mobile("Galaxy",2.2));
   Iterator itr=ll.iterator();
   while(itr.hasNext()){
      S.o.pln(itr.next());
   }
}
```

Name	PriorityQueue
Child of	Queue
Based on Data Structure	Priority Based Heap
Best suited operation	Priority Based Service

Heterogeneous-NO Insertion order-NO Default Capacity-11
Duplicates-yes Null Acceptance-NO

Marker Interface - Serializable, Cloneable
Version - V1.5
Package - java.util

```
Public class MainClass
{
   PSVM()
   {
     PriorityQueue pq = new PriorityQueue();
     pq.add("core java");
     pq.add("sql-Database");
     pq.add("web-Tech");
     pq.add("j2ee");
     pq.add("j2ee");
     pq.add("Frameworks");
     Iterator itr = pq.iterator();
     while(itr.hasNext()){
       S.O.Pln(itr.next());
     }
   }
}
```

Name	HashSet
Child of	Set
Based on Data Structure	Hashtable
Best suited operation	Searching

Heterogeneous-Yes Null Acceptance - Yes-Only
Duplicates - NO Default Capacity - 16
Insertion order - NO Load Factor/fill Ratio - 0.75F

Marker Interface - Serializable, Cloneable
Version - V1.82
Package - java.util

```
Public class MainClass
{
  PSVM(){
  HashSet hs = new HashSet();
  hs.add("java");
  hs.add(100);
  hs.add(null);
  hs.add(new Employee(120,4.5));
  hs.add(100);
  Iterator itr = hs.iterator();
  while(itr.hasNext()){
    S.O.Pln(itr.next());
  }
}
```

Name	LinkedHashSet
Child of	Set
Based on Datastructure	Hashtable+LinkedList
Best Suited Operation	Cache Based Applications
Heterogeneous - yes	Null Acceptance - yes-only
Duplicates - NO	Default Capacity - 16
Insertion order - yes	Load Factor/fill Ratio - 0.75F

Marker Interface - Serializable, Cloneable
Version - V1.4
Package - java.util

```
Public class MainClass
{
  PSVM(){
  LinkedHashSet hs = new LinkedHashSet();
  hs.add("Bangalore");
  hs.add("Chennai");
  hs.add(143);
  hs.add(null);
  hs.add(new Mobile("Iphone",18));
  Iterator itr = hs.iterator();
  while(itr.hasNext()){
    S.O.Pln(itr.next());
  }
}
```

Name	TreeSet
Child of	Set
Based on Datastructure	Balanced Tree
Best suited operation	Sorting

Heterogeneous	Duplicates	Insertion order	Null Acceptance
NO	NO	NO	NO

Marker Interface - Serializable, Cloneable
Version - V1.2
Package - java.util

```
Public class MainClass
{
 psvm(){
TreeSet t = new TreeSet();
  t.add(40);
  t.add(10);
  t.add(50);
  t.add(30);
  t.add(20);
Iterator itr = t.iterator();
while(itr.hasNext()){
  s.o.pln(itr.next());
}
}
```

O/P
10
20
30
40
50

HashSet, HashMap

- → Insertion order is not Preserved
- → HashTable
- → Searching
- → V1.2

LinkedHashSet & LinkedHashMap

- → Insertion order is Preserved.
- → HashTable + LinkedList
- → cache based applications
- → V1.4

	Heterogeneous	Duplicates	Insertion order	Null Acceptance
ArrayList	Yes	Yes	Yes	Yes
Vector	Yes	Yes	Yes	Yes
LinkedList	Yes	Yes	Yes	Yes
PriorityQueue	NO	Yes	NO	NO
HashSet	Yes	NO	NO	Yes-1
LinkedHashSet	Yes	NO	Yes	Yes-1
TreeSet	NO	NO	NO	NO

Name	HashMap
Child of	Map
Based on Datastructure	Hashtable
Best suited operation	Searching

Heterogeneous: Keys - Yes, Values - Yes
Duplicates: Keys - NO, Values - Yes
Insertion order: NO
Default Capacity: 16
Null Acceptance: Key - 1, Values - Yes
LoadFactor/Fill Ratio: 0.75F

Marker Interface - Serializable, Cloneable
Version - V1.2
Package - java.util

Name	LinkedHashMap
Child of	Map
Based on datastructure	Hashtable + LinkedList
Best suited operation	Cache based application

Heterogeneous: Keys - Yes, Values - Yes
Duplicates: Keys - NO, Values - Yes
Insertion order: Yes
Default Capacity: 16
Null Acceptance: Key - 1, Values - Yes
LoadFactor/FillRatio: 0.75F

Marker interface - Serializable, Cloneable
Version - V1.4
Package - java.util

Name	TreeMap
Child of	Map
Based on datastructure	Red-Black Tree
Best suited operation	Sorting
Heterogeneous: Keys-NO values-Yes	Duplicates: Keys-NO values-Yes
Insertion order: NO	Null Acceptance: Keys-NO values-Yes

Marker Interface - Serializable, Cloneable
Version - V1.2
Package - java.util

```
Public class MainClass
{
  PSVM(){
  TreeMap tm = new TreeMap();
  tm.Put("Hyderabad", "Telangana");
  tm.Put("Mumbai", "Maharashtra");
  tm.Put("Pondicherry", null);
  tm.Put("Delhi", null);
  tm.Put("Andaman", 100);
  tm.Put("Bangalore", "Karnataka");
  tm.Put("Chandigarh", "Punjab");
  tm.Put("Chandigarh", "Haryana");
  tm.Put("Srinagar", "Jammukashmir");
  tm.Put("Jammu", "Jammukashmir");
  Set s = tm.EntrySet();
  Iterator itr = s.iterator();
```

```
while(itr.hasNext())
{
  S.O.Pln(itr.next());
}
}
```

Collection

→ It is used to store a group of objects together.
→ It supports iterator

Map

→ It is used to store a group of objects together in KEY=VALUE format.
→ It does not supports iterator.

Cursors of Collection Framework Library:

The cursors of collection framework library to help the programmer to retrieve or traverse the elements stored inside a collection.

→ Java Provides the following 3 cursors:

1. Iterator
2. ListIterator
3. Enumerator

1. Iterator: Iterator is an interface in the collection framework library that can be used to retrieve the values stored in a collection.
→ Implementation of this interface is provided internally by all the collections.
→ The collections Provide implementation for Iterator Interface inside the iterator() method using anonymous inner class.
→ The Iterator implementation will have 3 methods:
1. hasNext(): It moves the cursor to the next location and checks if an object is available.
2. next(): It retrieves the element in the current location.
3. remove(): It removes the element from the current location.
→ It works in all collections and is considered as Collection Universal Cursor.
→ It traverses the collection in forward direction i.e, it is unidirectional cursor.

```
Public class Program
{
  psvm() {
  ArrayList ll = new ArrayList();
   ll.add("Java");
   ll.add("SQL");
   ll.add("web");
   ll.add(100);
   ll.add("j2ee");
   ll.add(200);
   ll.add("frameworks");
   S.O.Pln(ll);
   S.O.Pln("----");
   Iterator itr = ll.iterator();
   while(itr.hasNext()){
     Object obj = itr.next();
     if(obj instanceof Integer){
       itr.remove();
     }
   }
   S.O.Pln("------");
   S.O.Pln(ll);
  }
}
```

O/P

[Java, SQL, web, 100, j2ee, 200, frameworks]

[Java, SQL, web, j2ee, frameworks]

2. ListIterator: It is an interface of the Collection framework present in java.util package.

→ It is a child of Iterator Interface.

→ Implementation of ListIterator interface is provided by all List Type Collections, internally using an anonymous inner class inside the listIterator() method.

Methods of ListIterator:

hasNext()
next()
hasPrevious()
Previous()
remove()
add()

→ It is a bidirectional cursor.

→ It only works for List Type Collections.

```
Public class Program
{
  psvm() {
  ArrayList al=new ArrayList();
   al.add(100);
   al.add(400);
   al.add(500);
   al.add(200);
   al.add(300);
  ListIterator itr = al.listIterator();
  S.O.Pln("Forwards");
  while(itr.hasNext()){
    S.O.Pln(itr.next());
  }
  S.O.Pln("Backwards");
  while(itr.hasPrevious()){
    S.O.Pln(itr.previous());
  }
}
}
```

O/P

```
Forwards
100
400
500
200
300
Backwards
300
200
500
400
100
```

```
LinkedList ll=new LinkedList();
ll.add("apple");
ll.add("Banana");
ll.add(null);
ll.add(100);
ll.add("Mango");
S.O.Pln(ll);
ListIterator itr=ll.listIterator();
while(itr.hasNext()){
 Object obj=itr.next();
 if(obj instanceof Integer){
   itr.remove();
 }
 if(obj==null){
   itr.remove();
   itr.add("Grapes");
 }
}S.O.Pln(ll);
```

O/P

```
[apple, Banana, null, 100, Mango]
[apple, Banana, Grapes, Mango]
```

3. Enumeration:

→ It is an interface in java.util Package.
→ It is a legacy interface that was introduced in V1.0 to retrieve values from Vector.
→ Implementation of Enumeration is provided internally by Vector using an anonymous inner class inside the elements() method.
→ It contains the following methods:
   1. hasMoreElements()
   2. nextElement()
→ It is a uni directional cursor, only moves in forwards direction.
→ It only works for vector.

```
Public class Program
{
  psvm(){
   Vector v = new Vector();
   v.add("Pavan");
   v.add("Sameer");
   v.add("Akshay");
   v.add("Guru");
   v.add("Keshava");
   Enumeration e = v.elements();
   while(e.hasMoreElements()){
     S.o.pln(e.nextElement());
   }
  }
}
```

Iterator	ListIterator
1. It is a uni-directional cursor and only moves in forwards direction	1. It is a bi-directional cursor and moves both forwards & backwards
2. It is a Universal Cursor and works with all types of Collections.	2. It is not a universal cursor and only works for ListType Collections.
3. We can remove elements from Collections using Iterator	3. We can add and remove elements from Collections using ListIterator.

FOR-EACH / ADVANCED FOR LOOP / ENHANCED FOR LOOP

→ It was introduced in V1.5
→ It can be used by the programmer to retrieve the values stored inside the array (or) Collection.

```
Public class Program
{
  psvm(){
   int[] arr = {10,25,40,35,45,50};
   for(int a: arr)
   {
     S.o.pln(a);
   }
   double[] vals = {2.3,1.7,0.9,9.6};
   for(double val: vals)
   {
     S.o.pln(val);
   }
   String[] str = {"QSpiders","JSpiders","PySpiders","Praspiders"};
   for(String s: str)
   {
     S.o.pln(s);
   }
  }
}
```

Syntax: for(datatype varName : array/Collection) {
    } // instruction.

```
Public class Program
{
  Psvm(){
  HashSet hs = new HashSet();
  hs.add("Java");
  hs.add(100);
  hs.add(null);
  for(Object obj: hs){
    S.o.Pln(obj);
  }
}
```

GENERICS

→ Generics was introduced in java V1.5.
→ Using generics, the programmer can control what type of objects can be stored inside a collection.

```
Public class Program
{
  Psvm(){
  ArrayList<String> al = new ArrayList<String>();
  al.add("Java");
  al.add("Sql");
  al.add(100); → Integer is not allowed
  al.add("web");
  for(String s:al)
  {
    S.o.Pln(s);
  }
}
```

```
LinkedHashSet<Product> lhs = new LinkedHashSet();
lhs.add(new Product(101, 4500.0));
lhs.add(new Product(102, 9400.0));
lhs.add(new Product(103, 6200.0));
lhs:add(new Product(104, 5500.0));
for(Product P:lhs)
{
  S.o.Pln(P);
}
```

```
HashMap<String, Integer> hm = new HashMap<String, Integer>();
hm.Put("Bangalore", 29);
hm.Put("Chennai", 33);
hm.Put("Hyderabad", 25);
Set S = hm.entrySet();
for(Object o:S){
  S.o.Pln(o);
}
```

COLLECTION INTERFACE:

→ It is a Pre-defined interface Present in java.util Package
→ It was introduced in V1.2
→ It is considered as the root interface of the Collection hierarchy.
→ The Collection interface defines several methods that can be used by the Programmers in all the collections to perform basic CRUD operations

1. add()

→ The Programmer can use add() method to insert an object into the collection
→ The add() method returns true/false to indicate the result.

Ex.
```
LinkedHashSet lhs = new LinkedHashSet();
S.o.Pln(lhs.add("Java")); // true
S.o.Pln(lhs.add("Java")); // false
S.o.Pln(lhs.add("J2ee")); // true
S.o.Pln(lhs.add("Spring")); // true
S.o.Pln(lhs); // [Java, J2ee, Spring]
```

2. addAll()

```
ArrayList al = new ArrayList();
al.add("Red");
al.add("yellow");
al.add("Green");
LinkedList ll = new LinkedList();
ll.add("stop");
ll.add("wait");
ll.add("Go");
HashSet hs = new HashSet();
S.o.pln(hs.addAll(al)); //true
S.o.pln(hs.addAll(ll)); //true
S.o.pln(hs); // [Red, wait, Green, yellow, stop, wait]
```

→ The programmer can use addAll() method to merge multiple Collections into a single Collection

→ It returns true/false to indicate the result.

3. size()

→ The size() method returns the no.of elements present in the current collection.

Ex:
```
PriorityQueue<String> pq = new PriorityQueue<String>();
S.o.pln(pq.size()); //0
pq.add("JSpiders");
pq.add("QSpiders");
S.o.pln(pq.size()); //2
pq.add("PySpiders");
pq.add("ProSpiders");
S.o.pln(pq.size()); //4
```

4. isEmpty()

→ The programmer can use isEmpty() method to check if a collection is empty or having objects.

```
TreeSet ts = new TreeSet();
S.o.pln(ts.isEmpty()); // true
ts.add(23);
ts.add(17);
ts.add(24);
S.o.pln(ts.isEmpty()); // false
S.o.pln(ts); // [17, 23, 24]
```

5. Contains()

→ The contains() method is used to check if the given object is present in the present Collection.

→ It returns true/false to indicate the result.

Ex:
```
Vector v = new Vector();
v.add("Bangalore");
v.add("Chennai");
v.add("Mumbai");
v.add("Cochin");
S.o.pln(v.contains("Bangalore")); // true
S.o.pln(v.contains("Delhi")); // false
```

6. ContainsAll()

→ The ContainsAll() method is used to check if the current Collection contains the objects of the given Collection. It returns true/false to indicate the result.

Ex:
```
HashSet hs = new HashSet();
hs.add("CSE");
hs.add("ISE");
hs.add("ECE");
hs.add("EEE");
LinkedHashSet lhs = new LinkedHashSet();
lhs.add("MEC");
lhs.add("CIV");
lhs.add("TEL");
lhs.add("CHE");
S.o.pln(lhs.containsAll(hs)); // false
lhs.addAll(hs);
S.o.pln(lhs.containsAll(hs)); // true
```

7. remove()

→ We can use the remove() method to remove the given object from the current Collection.

→ It returns true/false to indicate the result.

Ex:
```
LinkedList ll = new LinkedList();
ll.add("Biryani");
ll.add("Pizza");
ll.add("Burger");
ll.add("Momos");
S.o.pln(ll.remove("Burger")); //true
S.o.pln(ll.remove("Chips")); // false
```

8. removeAll()

→ We can use the removeAll() method to removes all the objects of the given Collection from the current Collection.

→ It returns true/false to indicate the result.

Ex:
```
HashSet hs = new HashSet();
hs.add("CSE");
hs.add("ISE");
hs.add("ECE");
LinkedHashSet lhs = new LinkedHashSet();
lhs.add("MEC");
lhs.add("CIV");
lhs.add("TEL");
lhs.addAll(hs);
S.o.pln(lhs);
lhs.removeAll(hs);
S.o.pln(lhs);
```

9. retainAll()

→ The retainAll() method can be used to retain all the elements of given Collection while removing elements which are not present in the given Collection.

```
ArrayList al = new ArrayList();
al.add("Java");
al.add("SQL");
al.add("web");
al.add("J2EE");
al.add("Frameworks");
LinkedList ll = new LinkedList();
ll.add("Aptitude");
ll.add("Programming");
ll.add("Verbal");
ll.addAll(al);
S.o.pln(ll);
ll.retainAll(al);
S.o.pln(ll);
```

10. clear()

→ The clear() method will remove all the objects from the current Collection.

Ex:
```
Treeset ts = new Treeset();
ts.add("Sameer");
ts.add("Akshay");
ts.add("Pavan");
ts.add("Aparna");
S.o.pln(ts); // [SameerAparna, Akshay, Pavan, Sameer]
ts.clear();
S.o.pln(ts); []
```

There is no concrete class that directly implements Collection interface.

List Interface

→ The List interface is a child of Collection interface and is present in java.util Package

→ The implementation classes of List interface provide ordered Collections which preserves insertion order, allows duplicates, heterogeneous & null values.

→ The List interface specifies a few methods that can be used by all List type Collections.

1. add(int index, Object O)

```
ArrayList al = new ArrayList();
al.add("Bangalore");
al.add("Chennai");
al.add("Hyderabad");
S.o.pln(al);
al.add(1, "Mumbai");
S.o.pln(al); // [Bangalore, Mumbai, Chennai, Hyderabaid]
al.add(2, "Delhi");
S.o.pln(al); // ["Bangalore, Mumbai, Delhi, Chennai, Hyderabaid]
```

2. remove(int index)

The remove() method of List interface can be used to remove a particular element from the given index value.

Ex:
```
LinkedList ll = new LinkedList();
ll.add("Black");
ll.add("Blue");
ll.add("Red");
ll.add("white");
ll.add("yellow");
S.o.pln(ll); [Black, Blue, Red, white, yellow]
ll.remove(2);
S.o.pln(ll); [Black, Blue, white, yellow]
```

3. get(int index)

→ The get() method can be used to retrieve an element from the specified index in the given Collection.

→ If index is invalid, it will cause Exception.

Ex:
```
Vector v = new Vector();
v.add("Infosys");
v.add("TCS");
v.add("HCL");
v.add("IBM");
v.add("Wipro");
S.o.pln(v.get(2)); // HCL
S.o.pln(v.get(3)); // IBM
```

4. set(int index, Object O)

→ The set() method can be used by the Programmers to replace the current element at the specified index with the given element.

Ex:
```
ArrayList al = new ArrayList();
al.add("Biryani");
al.add("Momos");
al.add("Pizza");
al.add("Ragi Balls");
S.o.pln(al); // [Biryani, Momos, Pizza, Ragi Balls]
al.set(2, "Pasta");
S.o.pln(al); // [Biryani, Momos, Pasta, Ragi Balls]
```

5. indexOf(Object O)

→ The indexOf() method returns the index value of the given object in the current Collection.

→ It searches left to right.

→ It returns "-1" if the object is not found.

→ The lastIndexOf() method works in the same manner as indexOf() method but searches from right to left.

Ex: List l = new ArrayList();
l.add("Pani Puri");
l.add("Bhel Puri");
l.add("Dahi Puri");
l.add("Pani Puri");
l.add("Masala Puri");
l.add("Dahi Puri");
S.o.pln(l.indexOf("Pani Puri")); //0
S.o.pln(l.lastIndexOf("Pani Puri")); //3
S.o.pln(l.indexOf("Chat Puri")); //-1

ListIterator() returns an implementation of ListIterator interface. Interface provided through an anonymous inner class.

Queue Interface

→ The Queue interface is a child of Collection interface and is present in java.util Package
→ the implementation classes of Queue interface are LinkedList and PriorityQueue.
→ The implementation of Queue interface are used to develop service based operations.
→ the Queue interface provides a few methods that can be used on Queue type objects.

offer()
This method can be used to insert the object into Queue

Peek()
The Peek() method returns the element at the head of the Queue.

Poll() It removes and returns the element at the head of the Queue

Ex:
```
LinkedList ll = new LinkedList();
ll.offer("Java");
ll.offer("Sql");
ll.offer("web");
ll.offer("j2ee");
S.o.pln("orginal :"+ll); // [Java,Sql,web,j2ee]
S.o.pln("Peek Operation :"+ll.Peek()); // [Java]
S.o.pln("After Peek :"+ll); // [Java, Sql, web, j2ee]
S.o.pln("Poll Operation :"ll.Poll()); // [Java]
S.o.pln("After Poll :"+ll); // [Sql, web, j2ee]
```

offerFirst(): It adds an element to the head of the Queue
offerLast(): It adds an element to the tail of the Queue
PeekFirst(): It returns head element of the Queue
PeekLast(): It returns tail element of the Queue
PollFirst(): It removes & returns the head element of the Queue
PollLast(): It removes & returns the tail element of the Queue

Set Interface

→ Set is a child of Collection interface and is present in java.util Package
→ The Set interface does not provide any new methods.
→ It contains the abstract methods declared in Collection interface.
→ The implementations of Set interface are best suited when the programmer wants to avoid duplicate values

Sorted Set Interface

→ The Sorted Set interface is a child of Set interface and is present in java.util Package.
→ Sorted Set interface provides several methods that can be used in Tree Set

first(): It returns the first element in the set.
last(): It returns the last element in the set.
headSet(): It returns all the objects above (or before) the given value.
tailSet(): It returns all the objects after the given object & it includes given object.
subSet(): It creates a sub set between the first object and the second object given as argument. the first object is included, the 2nd object is excluded.

Ex: TreeSet ts = new TreeSet();
ts.add(150);
ts.add(300);
ts.add(200);
ts.add(250);
ts.add(100);
S.o.pln(ts); // [100,150,200,250,300]
S.o.pln(ts.first()); 100
S.o.pln(ts.last()); 300
S.o.pln(ts.headSet(250)); [100,150,200]
S.o.pln(ts.tailSet(250)); [250,300]
S.o.pln(ts.subSet(150,250)); [150,200]

NavigableSet Interface

→ The NavigableSet interface is a child of SortedSet interface
→ It was introduced in V1.6 and it provides several methods related to navigation inside a TreeSet.

Ex: TreeSet ts = new TreeSet();
ts.add(30);
ts.add(20);
ts.add(10);
ts.add(50);
ts.add(40);
S.o.pln(ts); [10,20,30,40,50]
S.o.pln(ts.floor(30)); 30
S.o.pln(ts.lower(30)); 20
S.o.pln(ts.ceiling(30)); 30
S.o.pln(ts.higher(30)); 40
S.o.pln(ts.pollFirst()); 10
S.o.pln(ts); [20,30,40,50]
S.o.pln(ts.pollLast()); 50
S.o.pln(ts); [20,30,40]

floor(): It returns the next element which is less than or equal to given element.
lower(): It returns the next element which is less than the given element.
ceiling(): It returns the next element in the set which is greater than or equal to the given element.
higher(): It returns the next element which is greater than the given element.
pollFirst(): It returns and removes the first element of the set.
pollLast(): It returns and removes the last element of the set.
descendingSet():
It reverse the order in which elements are stored in a TreeSet.
descendingIterator(): It provides a reverse iterator that can be used to traverse the elements of the TreeSet in reverse order.

Ex: TreeSet ts = new TreeSet();
ts.add("Chennai");
ts.add("Bangalore");
ts.add("Hyderabad");
ts.add("Mumbai");
ts.add("Delhi");
ts.add("Pune");
S.o.pln(ts);
S.o.pln(ts.descendingSet());
S.o.pln
Iterator itr = ts.iterator();
while(itr.hasNext()){
S.o.pln(itr.next());
}
S.o.pln();
Iterator revItr = ts.descendingIterator();
while(revItr.hasNext()){
S.o.pln(revItr.next());
}

Map Interface

→ The Map interface is an independent interface of Collection framework library present in java.util Package.
→ The implementations of Map interface will help the Programmer to store objects in key-value format.
→ The Map interface declares several abstracts methods that can be used in all its implementation classes.

Put():
Put() method will insert one key-value pair into the Map.

PutAll(): The PutAll() method will take the key-value pairs of one Map and insert into the current Map.

Ex:
```
HashMap hm = new HashMap();
hm.Put("Bangalore", "Karnataka");
hm.Put("Amaravathi", "Andhra Pradesh");
hm.Put("Chennai", "Tamil Nadu");
LinkedHashMap lhm = new LinkedHashMap();
lhm.Put("Mumbai", "Maharashtra");
lhm.Put("Bhopal", "Madhya Pradesh");
lhm.Put("Patna", "Bihar");
lhm.PutAll(hm);
S.o.Pln(lhm);
```

get(): It returns the value associated with the given key.
→ If the given key is not available, it returns "null".

remove():
It removes a key-value pair from the Map.

containsKey():
Checks the Map to see if the given key is available or not.

containsValue(): Checks the Map to see if the given value is present or not.

Ex:
```
LinkedHashMap lhm = new LinkedHashMap();
lhm.Put(20, "Coffee");
lhm.Put(40, "Idly");
lhm.Put(50, "Rice Bath");
S.o.Pln(lhm.get(40)); // Idly
S.o.Pln(lhm.get(100)); // null
S.o.Pln(lhm.containsKey(20)); // true
lhm.remove(20);
S.o.Pln(lhm.containsKey(20)); // false
S.o.Pln(lhm.containsValue("Idly")); // true
lhm.remove(40);
S.o.Pln(lhm.containsValue("Idly")); // false
```

size(): It returns the no.of key-value pairs in the Map.

isEmpty(): we can use this method to check if the current Map is empty or it has any elements.

clear(): we can use this method to clear all the elements in the Map.

keySet(): It returns all the keys present in the Map.

values(): It returns all the values in the current Map.

entrySet(): This method merges the key and values into individual entry object and returns as a Set type object.

SortedMap:-

SortedMap is an interface that extends Map interface.
→ The SortedMap interface provides methods that are implemented in Tree Map.

firstKey(): It returns the key associated with the first entry in the TreeMap

lastKey(): It returns the key associated with the last entry in the TreeMap

headMap(): It creates a map from the first element.

tailMap(): It creates a map from the given key to the last element.

subMap(): It creates a map between the two given keys.

```
ex: TreeMap tm = new TreeMap();
    tm.put("Java", "Development");
    tm.put("Python", "AI&ML");
    tm.put("JS", "FrontEnd");
    tm.put("SQL", "Database");
    S.o.pln(tm);
    S.o.pln(tm.firstKey());
    S.o.pln(tm.lastKey());
    S.o.pln(tm.headMap("JS"));
    S.o.pln(tm.tailMap("JS"));
    S.o.pln(tm.subMap("Java", "Python"));
```

NavigableMap :-

→ NavigableMap is a child of SortedMap interface and is present in java.util package.

→ It was introduced in V1.6

floorKey(): It returns the key which is less than or Equal to given key.

floorEntry(): It returns the entry with key less than or Equal to given key.

lowerKey(): It returns the key which is less than the given key.

lowerEntry(): It returns the entry with key is less than the given key.

ceilingKey(): It returns the key which is greater than or Equal to given key.

ceilingEntry(): It returns the entry with key is greater than or equal to given key.

higherKey(): It returns the key which is greater than the given key.

higherEntry(): It returns the entry with key is greater than the given key.

```
ex: TreeMap tm = new TreeMap();
    tm.put(8.4, "Renu");
    tm.put(8.9, "Sai");
    tm.put(9.0, "Pradeep");
    tm.put(8.5, "Uma");
    S.o.pln(tm.floorKey(8.5));
    S.o.pln(tm.floorEntry(8.5));
    S.o.pln(tm.lowerKey(8.5));
    S.o.pln(tm.lowerEntry(8.5));
    S.o.pln(tm.ceilingKey(8.9));
    S.o.pln(tm.ceilingEntry(8.9));
    S.o.pln(tm.higherKey(8.9));
    S.o.pln(tm.higherEntry(8.9));
```

pollFirstEntry(): It returns and removes the first entry in the TreeMap.

pollLastEntry(): It returns and removes the last entry in the TreeMap.

descendingMap(): It reverses the order of entries in the current map.

descendingKeySet(): It creates the set with all the keys of the current map in reverse order.

Ex: TreeMap tm = new TreeMap();
tm.put(5.9, "Ram");
tm.put(5.8, "Raj");
tm.put(5.6, "Akhil");
tm.put(5.7, "Sai");
tm.put(5.91, "Pradeep");
S.o.pln(tm.descendingMap());
S.o.pln(tm.descendingkeyset());
S.o.pln(tm);
S.o.pln(tm.PollFirstEntry());
S.o.pln(tm);
S.o.pln(tm.PollLastEntry());
S.o.pln(tm);

Comparable Interface:
→ It is a Pre-defined interface in java library
→ It is present in java.lang package
→ It is a functional interface which contains one abstract method.
Public abstract int compareTo(Object obj)
→ we can use Comparable interface to define or create Comparable type objects.

Ex:
```
Public class Image implements Comparable<Image>
{
  int size;
  Public Image(int size){
    this.size=size;
  }
  @Override
  Public String toString(){
    return "Image size :" +size;
  }
  @Override
  Public int CompareTo(Image o){
    if(this.size<o.size){
      return -1;
    }else if(this.size>o.size){
      return +1;
    }else{
      return 0;
    }
  }
Public class MainClass
{ PSVM(){
  PriorityQueue<Image> Pq= new PriorityQueue<Image>();
  Pq.offer(new Image(2345));
  Pq.offer(new Image(2287));
  Pq.offer(new Image(5678));
  Pq.offer(new Image(1027));
  Pq.offer(new Image(3459));
  S.o.pln(Pq);
  while(Pq.isEmpty()==false){
    S.o.pln(Pq.Poll());
  }
}
}
```

```
Public class Student implements Comparable<Student>{
  String name;
  int yop;
  double cgpa;
  Public Student(String name, int yop, double cgpa){
    super();
    this.name=name;
    this.yop=yop;
    this.cgpa=cgpa;
  }
  @Override
  Public String toString(){
  return "[name="+name+ ",yop=" +yop+ ", cgpa=" +cgpa+ "]";
  }
```

```
@Override
Public int compareTo(Student o){
    if(this.cgpa < o.cgpa){
        return -1;
    }
    else if(this.cgpa > o.cgpa){
        return +1;
    }
    else{
        return 0;
    }
}}
Public class MainClass
{
    psvm(String[] args){
    TreeSet<Student> ts = new TreeSet<Student>();
    ts.add(new Student("Renu", 2025, 8.3));
    ts.add(new Student("Rama", 2024, 8.2));
    ts.add(new Student("Lucky", 2017, 6.6));
    ts.add(new Student("Sandy", 2023, 8.5));
    for(Student s:ts){
        s.o.pln(s);
    }
}
}
```

```
Public class Bottle implements Comparable<Bottle>{
  int capacity;
  Public Bottle(int capacity){
     super();
     this.capacity=capacity;
  }
  @Override
  Public String toString(){
   return "[capacity=" + capacity + "]";
  }
  @Override
  Public int compareTo(Bottle o){
     return this.capacity-o.capacity;
  }
}
```

```
Public class MainClass
{
  psvm(String[] args){
    TreeMap<Student> tm = new TreeMap<Student>
  TreeMap<Bottle,String> tm= new TreeMap<Bottle,String>();
    tm.put(new Bottle(2000), "Gym");
    tm.put(new Bottle(1000), "Class");
    tm.put(new Bottle(500), "Car");
    tm.put(new Bottle(5000), "Trek");
    s.o.pln(tm);
  }
}
```

Comparator Interface :-

→ It is a Pre-defined interface in the Collection framework library present inside java.util Package.

→ We can create implementation classes of Comparator to define Custom Sorting logic.

→ The Comparator interface contains the following two abstract methods.

1. int Compare(Object o1, Object o2)
2. boolean Equals(Object obj)

Ex:
```
Public class Product
{
    int Pid;
    double Price;
    double rating;
    Public Product(int Pid, double Price, double rating){
        super();
        this.Pid=Pid;
        this.Price=Price;
        this.rating=rating;
    }
    @Override
    Public String toString(){
     return "[Pid=" + Pid + ", Price=" + Price + ", rating=" +
                                                rating + "]";
    }
}
```

```
Public class PriceComparator implements
  @Override
  Public int compare(Product o1, Product o2)
  {
     if (o1.price < o2.price){
        return -1;
     }
     else if (o1.price > o2.price){
        return +1;
     }
     else{
        return 0;
     }
  }
}
Public class RatingComparator implements Comparator<Product>
{
  @Override
  Public int compare(Product o1, Product o2){
     if (o1.rating < o2.rating){
        return -1;
     }
     else if (o1.rating > o2.rating){
        return +1;
     }
     else{
        return 0;
     }
  }
}
Public class MainClass
{
  PSVM(String[] args){
  TreeSet<Product> ts = new TreeSet<Product>(new
                                RatingComparator());
     ts.add(new Product(101, 4500.0, 4.5));
     ts.add(new Product(103, 7800.0, 4.2));
     ts.add(new Product(104, 6200.0, 4.1));
     ts.add(new Product(102, 2900.0, 4.7));
     for (Product p:ts){
        S.O.println(p);
     }
  }
}
```

```
Public class Employee
{
  String name;
  int id;
  double ctc;
  Public Employee(String name, int id, double ctc){
    super();
    this.name = name;
    this.id = id;
    this.ctc = ctc;
  }
  @Override
  Public String toString(){
    return "[name=" + name + ", id=" + id + ", ctc=" + ctc + "]";
  }
}
Public class NameComparator implements Comparator<Employee>{
  @Override
  Public int compare(Employee o1, Employee o2){
    return o1.name.compareTo(o2.name);
  }
}
Public class SalaryComparator implements Comparator<Employee>{
  @Override
  Public int compare(Employee o1, Employee o2){
     if (o1.ctc < o2.ctc){
        return -1;
     }
     else if (o1.ctc > o2.ctc){
        return +1;
     }
     else{
        return 0;
     }
  }
}
Public class IdComparator implements Comparator<Employee>{
  @Override
  Public int compare(Employee o1, Employee o2){
     if (o1.id < o2.id){
        return -1;
     }
     else if (o1.id > o2.id){
        return +1;
     }
     else{
        return 0;
     }
  }
}
```